

Архитектура графического процессора

Прежде чем писать или настраивать ядра для графического процессора, вам нужна рабочая модель того, как графический процессор выполняет код. Без нее такие рекомендации, как «увеличить занятость» или «уменьшить конфликты в банке общей памяти», — это просто набор правил, которые нужно запомнить. Вы не до конца понимаете, когда они применимы, а когда нет.



В этом разделе объясняется современная архитектура графических процессоров на уровне, необходимом для работы с ядрами. Подробности касаются оборудования NVIDIA, поскольку CUDA сегодня доминирует в экосистеме логического вывода больших языковых моделей. Однако основные концепции применимы и к графическим процессорам AMD, и к другим параллельным ускорителям.

Аппаратное обеспечение и модель исполнения

Аппаратное обеспечение — это то, что физически присутствует в графическом процессоре: [потокные мультипроцессоры \(SM\)](#), [тензорные ядра](#), а также регистры, кэши, общая память и память с высокой пропускной способностью (HBM), которые образуют [иерархию памяти графического процессора](#). Эти ресурсы определяют, какой объем работы и данных графический процессор может хранить рядом с вычислительными блоками.

Модель выполнения — это абстракция, используемая для написания ядра. Код создает потоки и организует их в группы, блоки и сетки. Графический процессор распределяет эту работу между доступными аппаратными средствами.

[Оптимизация ядра](#) объединяет эти два подхода. Быстрое ядро сопоставляет модель выполнения с аппаратным обеспечением и учитывает ограничения по вычислительным ресурсам, пропускной способности памяти, регистрам, общей памяти и возможностям планирования.

Выполнение на графическом процессоре и карта памяти

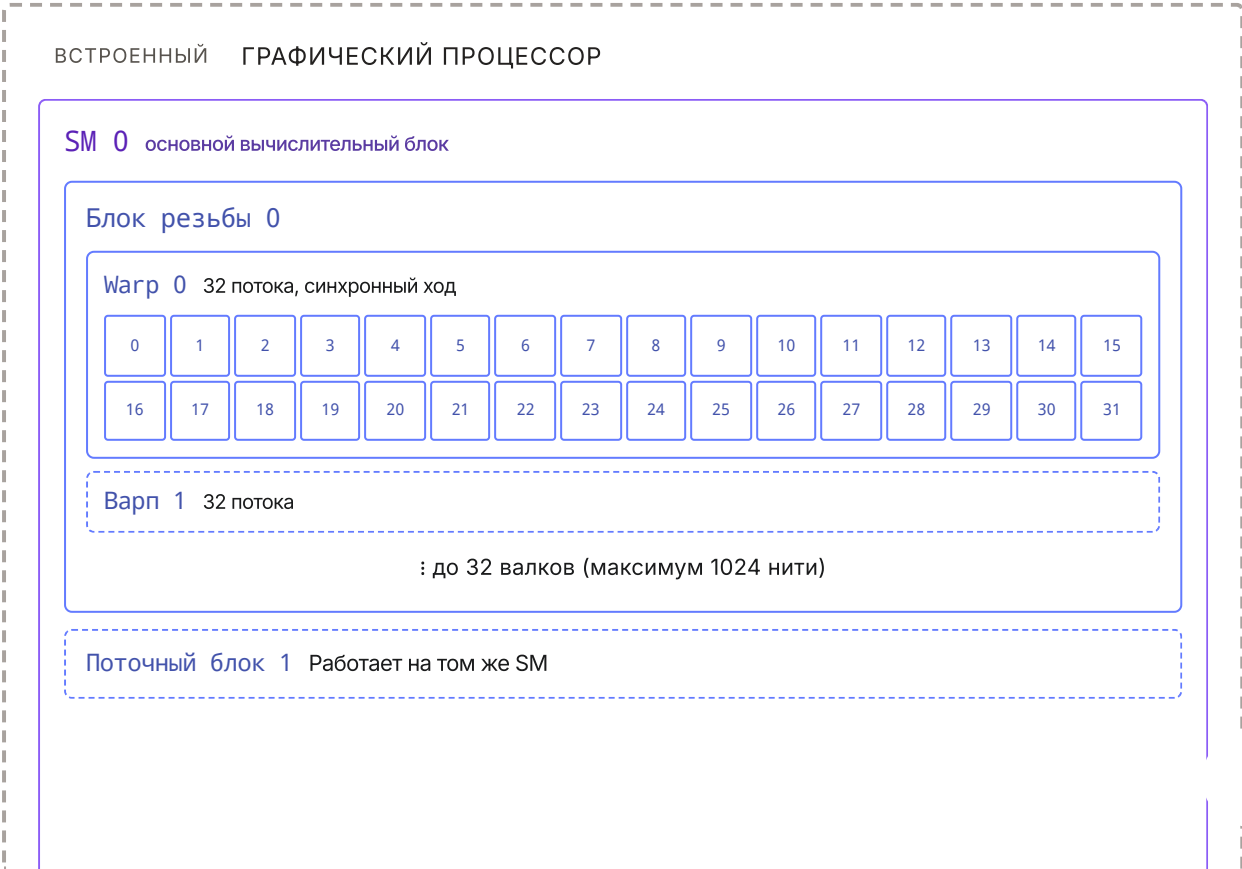
На рисунке ниже показано, как информация передается от отдельных потоков к супермашине, а затем — на уровни памяти, которые питают вычислительные блоки. Иерархия выполнения определяется четырьмя терминами:

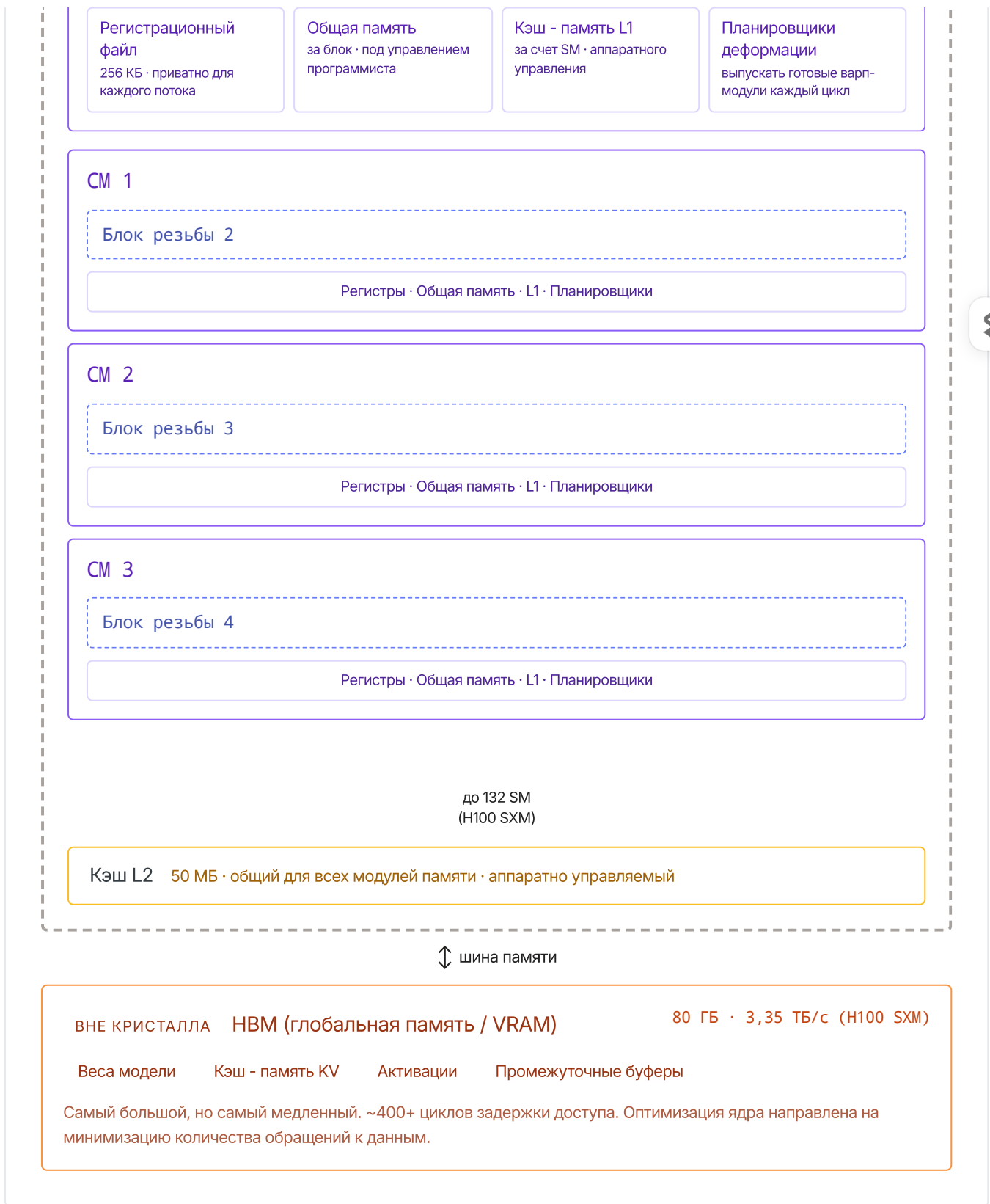


- [Поток](#) — это наименьшая логическая единица работы, которая обычно обрабатывает один элемент или небольшую группу элементов.
- [Группа](#) — это группа из 32 потоков, которые планировщик NVIDIA SM объединяет в одну группу.
- [Блок потоков](#) объединяет потоки, которые могут взаимодействовать через общую память и синхронизацию.
- [Потоковый мультипроцессор \(SM\)](#) — это встроенный вычислительный блок, который принимает блоки и выдает инструкции из готовых групп.

Выполнение на графическом процессоре и карта памяти

Узнайте, как организована работа и где хранятся данные — от отдельных потоков до глобальной памяти. Наведите курсор на любой раздел, чтобы выделить его.





Что дальше?

На четырех страницах ниже мы отделяем модель выполнения от физического оборудования, а затем показываем, как эти два представления пересекаются во время работы ядра.

Потоки, группы, блоки и сетки графического процессора

Узнайте, как ядра графического процессора распределяют работу между потоками, группами, блоками потоков и сетками.

Streaming multiprocessors

Understand streaming multiprocessor internals, warp scheduling, block residency, and GPU occupancy.



GPU memory hierarchy

Understand GPU registers, shared memory, caches, bank conflicts, HBM, and the trade-offs across the memory hierarchy.

Tensor Cores

Learn how Tensor Cores execute tiled matrix operations and what precision, shape, and layout constraints kernels must satisfy.

 [Edit this page](#)

 [Copy as Markdown](#)

Modular

[Models](#) [Blog](#) [Community](#) [GitHub](#) [Contact](#) [Privacy settings](#)